Coinsult

Request your audit at **coinsult.net**

Advanced Manual

Smart Contract Audit

October 30, 2026

x.com/coinsultaudits

t.me/coinsult_tg

Audit requested by

FOBO Ecosystem

https://github.com/fobodotfun/src/ (27 of 27 files reviewed)

Global Overview

Manual Code Review

In this audit report we will highlight the following issues:

Vulnerability Level	Total	Pending	Acknowledged	Resolved
Informational	8	8	0	0
Low-Risk	9	9	0	0
Medium-Risk	2	2	0	0
Critical-Risk	0	0	0	0

Risk Classification

Coinsult uses certain vulnerability levels, these indicate how bad a certain issue is. The higher the risk, the more strictly it is recommended to correct the error before using the contract.

Vulnerability Level	Description
Informational	Does not compromise the functionality of the contract in any way
Low-Risk	Won't cause any problems, but can be adjusted for improvement
Medium-Risk	Will likely cause problems and it is recommended to adjust
Critical-Risk	Will definitely cause problems, this needs to be adjusted

Audit Summary

FOBO	
Website	fobo.fun
Blockchain	Robinhood Chain (chain ID 4663)
Source Code	https://github.com/fobodotfun
Contract Address	-

This audit report has been prepared by Coinsult's experts at the request of the client. In this audit, the results of the static analysis and the manual code review will be presented. The purpose of the audit is to see if the functions work as intended, and to identify potential security issues within the smart contract.

The information in this report should be used to understand the risks associated with the smart contract. This report can be used as a guide for the development team on how the contract could possibly be improved by remediating the issues that were identified.

Audit Scope

Coinsult was commissioned to perform an audit based on the provided code.

Note that we only audited the code available to us on this URL at the time of the audit. If the URL is not from any block explorer (main net), it may be subject to change. Always check the contract address on this audit report and compare it to the token you are doing research for.

Audit Method

Coinsult's manual smart contract audit is an extensive methodical examination and analysis of the smart contract's code that is used to interact with the blockchain. This process is conducted to discover errors, issues and security vulnerabilities in the code in order to suggest improvements and ways to fix them.

Automated Vulnerability Check

Coinsult uses software that checks for common vulnerability issues within smart contracts. We use automated tools that scan the contract for security vulnerabilities such as integer-overflow, integer-underflow, out-of-gas-situations, unchecked transfers, etc.

Manual Code Review

Coinsult's manual code review involves a human looking at source code, line by line, to find vulnerabilities. Manual code review helps to clarify the context of coding decisions. Automated tools are faster but they cannot take the developer's intentions and general business logic into consideration.

Used tools

- Slither: Solidity static analysis framework
- Remix: IDE Developer Tool
- CWE: Common Weakness Enumeration
- SWC: Smart Contract Weakness Classification and Test Cases
- DEX: Testnet Blockchains

Table of Contents

Global Overview 2
- Manual Code Review 2
- Risk Classification 2

Audit Summary 3

Audit Scope 4
- Audit Method 4
- Automated Vulnerability Check 4
- Manual Code Review 4
- Used tools 4

Table of Contents 5
- The Hook Has No Owner, No Upgrade Path And No Way To Withdraw Pool Liquidity 6
  - Code Snippet 6
  - Recommendation 6
- The Launched Token Is A Plain Fixed Supply ERC-20 7
  - Code Snippet 7
  - Recommendation 7
- Scope And Method Of This Review 8
  - Code Snippet 8
  - Recommendation 8
- The Floor Price Is Monotone And The Absorb Guarantee Is Re-Checked After Every Change 9
  - Code Snippet 9
  - Recommendation 9
- The Fee Cap On A Limit Order Applies Per Fill, Not To The Order As A Whole 10
  - Code Snippet 10
  - Recommendation 10
- The Exit Guarantee At The Floor Holds But Carries No Explicit Margin 11
  - Code Snippet 11
  - Recommendation 11
- The Locker Cannot Be Touched By Anyone And The Buyback Is Bounded Against Sandwiching 12
  - Code Snippet 12
  - Recommendation 12
- The Declared Surface Matches The Code And Nothing Is Left Dangling 13
  - Code Snippet 13
  - Recommendation 13
- The Creator's Launch Buy Is Exempt From The Tax Every Other Early Buyer Pays 14
  - Code Snippet 14
  - Recommendation 14
- Every Launch After The First Rests On One Hot Signing Key 15
  - Code Snippet 15
  - Recommendation 15
- A Floor Raise That Fails Is Swallowed And The Block Is Marked Done Anyway 16
  - Code Snippet 16
  - Recommendation 16
- A Swap Reverts Unless It Carries Roughly 1.2 Million Gas 17
  - Code Snippet 17
  - Recommendation 17
- The Referral On A Swap Is Credited To tx.origin 18

Code Snippet 18
Recommendation 18
• Floor Money Beyond What Holders Can Carry Leaves The Token And Buys $FOBO 19
Code Snippet 19
Recommendation 19
• A Compromised Registrar Key Can Divert A Quarter Of The Protocol Fee 20
Code Snippet 20
Recommendation 20
• The Price Feed Is Checked For Staleness But Not For A Wrong Answer 21
Code Snippet 21
Recommendation 21
• A Limit Order Fills At Its Limit And The Keeper Keeps Everything Above It 22
Code Snippet 22
Recommendation 22
• A Quote Token Is Admitted On The Owner's Judgement With No On Chain Check 23
Code Snippet 23
Recommendation 23
• The Owner Can Withdraw Every Launch Fee And Sweep Held For The $FOBO Buyback 24
Code Snippet 24
Recommendation 24
Disclaimer 25

The Hook Has No Owner, No Upgrade Path And No Way To Withdraw Pool Liquidity

Informational *- Does not compromise the functionality of the contract in any way*
[Status: Pending]

Error Code	Description
FOBO-I1	The claim that a FOBO pool's liquidity cannot be removed was checked rather than taken on trust. The hook inherits no ownership contract and carries no privileged function, there is no delegatecall, proxy or selfdestruct anywhere in the source, and the three callbacks that would let an outsider create a pool on this hook, add liquidity or donate are deliberately left reverting. The factory does have an owner, and its powers reach only future launches: pausing, fees, bands, the allowed quotes and the signer, with ownership two step and renouncing disabled.

Code Snippet

```
// src/FoboHook.sol:34   contract FoboHook is HookClaims, IUnlockCallback
//                       no Owned, no onlyOwner anywhere in src/hook/

// src/base/BaseHook.sol:36  enabled only in order to refuse
function beforeInitialize(...) external virtual returns (bytes4) {
    revert Disabled();
}

// src/base/Owned.sol:13  the factory owner cannot walk away
function renounceOwnership() public pure override {
    revert RenounceDisabled();
}
```

Recommendation

No change needed. Keep the hook free of any owner or upgrade mechanism, since this property is the basis of the product's main claim.

The Launched Token Is A Plain Fixed Supply ERC-20

Informational *- Does not compromise the functionality of the contract in any way*
[Status: Pending]

Error Code	Description
FOBO-I2	Each launch deploys an unmodified OpenZeppelin ERC-20 with a fixed supply of one billion minted once to the factory, which passes the whole amount to the hook as pool liquidity. There is no owner, no mint function, no transfer fee, no pause and no blocklist, so the supply cannot grow and no address can be frozen after launch. Holders can burn their own tokens, which the floor treats as permanently gone.

Code Snippet

```
// src/FoboToken.sol:12
contract FoboToken is ERC20, ERC20Burnable, IFoboToken {
    uint256 public constant TOTAL_SUPPLY = 1_000_000_000e18;
    constructor(string memory name_, string memory symbol_)
        ERC20(name_, symbol_) {
        _mint(msg.sender, TOTAL_SUPPLY);
    }
```

Recommendation

No change needed.

Scope And Method Of This Review

Informational *- Does not compromise the functionality of the contract in any way*
[Status: Pending]

Error Code	Description
FOBO-I3	This review covers the twenty seven Solidity files FOBO wrote, read line by line, alongside the deployed hook's verified source. Uniswap v4 core and OpenZeppelin are treated as trusted third party code and sit outside the scope: their behaviour is taken as specified, and what is checked is how FOBO uses them rather than whether they are correct in themselves. Two limits of the method are worth stating plainly. No test suite was run, so the floor's arithmetic was confirmed by tracing every path that writes it rather than by fuzzing it, and the deployment scripts together with the on chain configuration they set are not part of this scope.

Code Snippet

```
// in scope: 27 files, all of FOBO's own code
src/                7 contracts
src/hook/           4 hook modules
src/base/           2 and src/libraries/ 6 helpers
src/interfaces/     8 interfaces

// out of scope: trusted and assumed correct
v4-core (Uniswap v4)
@openzeppelin/contracts
```

Recommendation

A fuzz run over the floor invariant, as FOBO-I6 sets out, would let a later review confirm by test what this one confirms by reading.

The Floor Price Is Monotone And The Absorb Guarantee Is Re-Checked After Every Change

Informational *- Does not compromise the functionality of the contract in any way*
[Status: Pending]

Error Code	Description
FOBO-I4	The claim that the floor only moves up was traced rather than assumed. The search for a new floor is bounded above by the current floor less one tick spacing, so the stored floor tick can only move in the direction of a higher price, and there is no path anywhere that moves it the other way. Both the raise and the sweep end by recomputing what the positions actually hold against the circulating supply and reverting with FloorBroken if the floor could no longer buy every token back, and either path rolls back on its own without stopping trading.

Code Snippet

```
// src/hook/HookFloor.sol:107   the search can never go below the floor
c.tHigh = tickF - SPACING;

// src/hook/HookFloor.sol:262   after a raise
if (_absorbNow(p, sNow) < _circulating(p, sNow)) revert FloorBroken();

// src/hook/HookFloor.sol:289   after a sweep
if (_absorbNow(p, c.sCur) < c.circulating) revert FloorBroken();
```

Recommendation

No change needed.

● The Fee Cap On A Limit Order Applies Per Fill, Not To The Order As A Whole

Informational *- Does not compromise the functionality of the contract in any way*
[Status: Pending]

Error Code	Description
FOBO-I5	An order carries a maximum fee per fill, and nothing limits how many fills an order can be split into, so the total a keeper can take is that figure multiplied by however many fills it chooses to make rather than the single number the owner set. What actually protects the owner is the all-in limit check, which counts the fee as part of what was spent and so refuses a fill whose fee ate the price, and the protection holds. The name is still likely to be read as a cap on the whole order.

Code Snippet

```
// src/FoboLimitOrders.sol:170  per fill, not per order
if (fee > order.maxFeePerFill || (buy && fee >= amountIn))
    revert FeeTooHigh();
```

Recommendation

Label it as a per fill cap wherever the order is placed, and consider tracking fees paid against a per order total so the figure the owner sets is the figure they pay.

The Exit Guarantee At The Floor Holds But Carries No Explicit Margin

Informational *- Does not compromise the functionality of the contract in any way*
[Status: Pending]

Error Code	Description
FOBO-I6	The band of quote under the price is the last bid in the pool, since no liquidity exists above its far edge, so whether a holder can always sell rests entirely on the check that the band can absorb the whole circulating supply. That check is set up conservatively, with the supply figure rounded up against the pool and the capacity figure rounded down, so the real capacity exceeds the real float and the last holder can get out. There is no margin beyond that rounding, and because a swap must fill completely or revert, a seller who asks for more than the pool can give is refused outright rather than filled in part.

Code Snippet

```
// src/hook/HookFloor.sol:262  the guarantee
if (_absorbNow(p, sNow) < _circulating(p, sNow)) revert FloorBroken();

// src/hook/HookTax.sol:97  a sell must fill completely
if (delta.amount1() != params.amountSpecified) revert PartialFill();
```

Recommendation

Add a fuzz test that sells the entire circulating supply into the floor, once as a single swap and once as many small ones, and assert that neither reverts; then size the band with a small explicit margin so the guarantee does not rest on rounding direction alone.

The Locker Cannot Be Touched By Anyone And The Buyback Is Bounded Against Sandwiching

Informational *- Does not compromise the functionality of the contract in any way*
[Status: Pending]

Error Code	Description
FOBO-I7	Two things worth recording as they stand. The vesting contract has no owner, no upgrade path and no early exit, it records what actually arrived rather than what was requested so a token that takes a cut on transfer is locked correctly, and anyone can trigger a release but it only ever pays the beneficiary. The buyback's $FOBO purchase is bounded against the pool's tick at the start of the block rather than the price just before the swap, so an attacker who moves the price first uses up the allowance themselves, and what they could still gain is below the tax they would pay going in and out.

Code Snippet

```
// src/FoboVesting.sol:38  locks what actually arrived
uint256 received = IERC20(token).balanceOf(address(this)) - before;

// src/FoboBurner.sol:141  bounded from the block's start
uint160 sBound = TickMath.getSqrtPriceAtTick(
    h.startOfBlockTick(id) - BURN_MAX_TICKS);
if (sNow < sBound) revert BurnBound();
```

Recommendation

No change needed.

The Declared Surface Matches The Code And Nothing Is Left Dangling

Informational *- Does not compromise the functionality of the contract in any way*
[Status: Pending]

Error Code	Description
FOBO-I8	Every contract inherits the interface that describes it, the hook included, so the surface integrators build against is enforced by the compiler rather than kept in step by hand. Across all twenty seven files there is no error declared that cannot be reached and no event declared that is never emitted, so the revert reasons and the monitoring surface the interfaces document are the ones the deployed code actually produces. The interface comments were checked against behaviour on the points that carry weight, including that a pay token's pricing is copied into an order at placement so a later owner change cannot reach an open one, and that the burner's withdrawal power is stated plainly where it is declared rather than buried.

Code Snippet

```
// src/hook/HookStorage.sol:24  the hook's surface is enforced
abstract contract HookStorage is BaseHook, IFoboHook {

// src/interfaces/IFoboBurner.sol:41  stated where it is declared
/// @notice Sends everything held of `token`, its ERC-6909 claims
///         redeemed, to `to`. Owner only.

// 0 unused errors and 0 unemitted events across src/
```

Recommendation

No change needed.

● The Creator's Launch Buy Is Exempt From The Tax Every Other Early Buyer Pays

Low-Risk *- Won't cause any problems, but can be adjusted for improvement*
[Status: Pending]

Error Code	Description
FOBO-L1	For the first sixty seconds of a pool the swap tax starts at ninety percent and falls to two and a half, which is what stops a sniper taking the launch. The creator's own buy, made by the factory inside the launch transaction, is excluded from that curve and pays the base two and a half percent on up to five percent of the supply. The exclusion is correctly scoped, since it only covers the factory on the pool being launched and is cleared when the launch ends, and it does hand the creator an advantage over every other buyer in the same block that the public description of the launch tax does not mention.

Code Snippet

```
// src/FoboHook.sol:92  createPool marks the pool as launching
bytes32 slot = LAUNCHING_SLOT;
assembly ("memory-safe") { tstore(slot, id) }

// src/hook/HookTax.sol:114  _rate
if (sender == factory) {
    if (launching == PoolId.unwrap(id)) return (BASE_TAX, false);
}
return Tax.rate(block.timestamp - p.launchedAt);  // 90% -> 2.5%
```

Recommendation

State plainly in the public documentation that the creator's launch buy pays the base rate rather than the launch tax, so buyers can price that in.

Every Launch After The First Rests On One Hot Signing Key

Low-Risk *- Won't cause any problems, but can be adjusted for improvement*
[Status: Pending]

Error Code	Description
FOBO-L2	Apart from the owner's own first launch, a token can only be created with a signature from the start signer, which the code comment describes as a hot key on the site's server. That signature is what binds a launch to the name, ticker and start price the site approved, so anyone holding the key can create launches that never passed the site's checks, including tokens named to impersonate something else, for the cost of the launch fee. The start band bounds the price such a launch can open at, and rotating the key voids outstanding signatures, but there is no second approval and no delay on the key itself.

Code Snippet

```
// src/FoboFactory.sol:188  _useStart
if (err != ECDSA.RecoverError.NoError || signer != startSigner)
    revert BadStartSignature();

// src/FoboFactory.sol:298  setStartSigner
function setStartSigner(address signer) external onlyOwner { ... }
```

Recommendation

Keep the signer key in a hardware signer or a service that enforces the same checks the site does, monitor TokenLaunched for launches the site did not originate, and rotate on any doubt.

● A Floor Raise That Fails Is Swallowed And The Block Is Marked Done Anyway

Low-Risk *- Won't cause any problems, but can be adjusted for improvement*
[Status: Pending]

Error Code	Description
FOBO-L3	The raise runs inside a try and catch so that a fault in it can never stop people trading, which is the right choice. The cost is that a raise which reverts only emits an event, and the block is still recorded as settled, so nothing retries until the next block and nothing on chain forces anyone to notice. Since the floor rising is the product's central promise, it can stop working while swaps carry on looking normal, and every block's first swapper keeps paying for the failed attempt.

Code Snippet

```
// src/hook/HookTax.sol:40  beforeSwap
if (p.lastRaiseBlock != bn) {
    if (_plan(p, id).go) {
        try this.raiseFromHook{gas: RAISE_GAS}(key) {}
        catch (bytes memory reason) { emit RaiseFailed(id, reason); }
    }
    p.lastRaiseBlock = uint64(bn);   // set even when the raise failed
}
```

Recommendation

Alert on every RaiseFailed event and treat a repeat on the same pool as an incident, since nothing else surfaces it.

A Swap Reverts Unless It Carries Roughly 1.2 Million Gas

Low-Risk *- Won't cause any problems, but can be adjusted for improvement*
[Status: Pending]

Error Code	Description
FOBO-L4	Before it will run the floor raise or the fee split, the hook checks that the transaction still holds the whole budget for each and reverts with NeedGas if not. Reserving the gas up front is deliberate, so that a wallet's estimate includes the work rather than quietly skipping it, and it does mean a swap can need about 1.2 million gas on a block where both paths fire and far less on every other block. A router or aggregator that sizes its gas from a simulation taken on a quiet block will have its transactions revert on the block that matters, and the same applies to a holder trying to sell at the floor on a busy day, since the block's first seller is the one who pays for the raise.

Code Snippet

```
// src/hook/HookTax.sol:42
if (gasleft() < RAISE_GAS * 64 / 63 + 40_000) revert NeedGas();
// src/hook/HookTax.sol:184
if (gasleft() < CREDIT_GAS * 64 / 63 + 20_000) revert NeedGas();

// src/hook/HookStorage.sol:67  RAISE_GAS  = 700_000
// src/hook/HookStorage.sol:72  CREDIT_GAS = 400_000
```

Recommendation

Document the worst case gas for a swap on a FOBO pool in the integration notes, and tell integrators to estimate on a block where the raise fires rather than taking the cheapest simulation.

The Referral On A Swap Is Credited To tx.origin

Low-Risk *- Won't cause any problems, but can be adjusted for improvement*
[Status: Pending]

Error Code	Description
FOBO-L5	Unless the swap came from the limit order contract, the hook credits the referral to the transaction's origin, which for anyone using a smart contract wallet or a bundled transaction is the bundler rather than the person trading. A referred user who swaps through such a wallet silently loses their referrer's share, which goes to the team instead, and if a bundler's own address is ever bound to a referrer then every swap it relays credits that referrer. Referrers are bound once by the registrar and cannot sign themselves up, which keeps this to misattribution rather than theft.

Code Snippet

```
// src/hook/HookTax.sol:198  _trader
if (sender == limitOrders) return abi.decode(hookData, (address));
return tx.origin;

// src/FoboRevenue.sol:93  the referrer is looked up from that address
address referrer = referrerOf[trader];
```

Recommendation

Let a router or wallet pass the trader in hookData the way the limit order contract does, from a list of senders the owner trusts, and fall back to the transaction origin only for direct swaps.

● Floor Money Beyond What Holders Can Carry Leaves The Token And Buys $FOBO

Low-Risk *- Won't cause any problems, but can be adjusted for improvement*
[Status: Pending]

Error Code	Description
FOBO-L6	A pool's floor only ever holds what its circulating tokens could sell into at the next tranche's price, and anything above that is swept out. Where it goes depends on the pool: a launched token's surplus buys and burns $FOBO, so tax paid by buyers of one token supports a different one, while $FOBO's own pool has no creator and its surplus is booked to the team as revenue instead. The guarantee that every circulating token can still sell into the floor is preserved and re-checked either way, and the published description of the floor does not mention that a surplus leaves the pool at all.

Code Snippet

```
// src/hook/HookFloor.sol:174  _planDrain
// floor money beyond what the circulating tokens can carry leaves

// src/hook/HookStorage.sol:168  _burnShare
if (p.creator == address(0)) {
    teamOwed[quote] += amount;   // $FOBO's own pool: to the team
} else if (q == fobo) {
    IFoboToken(q).burn(amount);  // a launched token: burns $FOBO
}
```

Recommendation

Say in the public description of the floor that a surplus beyond what the circulating supply can absorb leaves the pool, and that in $FOBO's own pool it becomes team revenue rather than a buyback.

● A Compromised Registrar Key Can Divert A Quarter Of The Protocol Fee

Low-Risk *- Won't cause any problems, but can be adjusted for improvement*
[Status: Pending]

Error Code	Description
FOBO-L7	One address, the site's relayer, decides which trader is credited to which referrer, and a referrer currently earns a quarter of the protocol fee on that trader's swaps. If that key is taken, the holder can bind every trader who has no referrer yet to their own address and collect on all of them until the key is rotated. Recovery works and was clearly designed for, but it is rotate the key and then clear the bindings two hundred traders at a time, and anything already credited stays claimable by whoever it was booked to.

Code Snippet

```
// src/FoboRevenue.sol:151  registerBy
if (msg.sender != registrar) revert NotRegistrar();
referrerOf[t] = r;   // set once, never changes after that

// src/FoboRevenue.sol:95  what that is worth
if (referrer != address(0))
    toReferrer = protocolFee * referralBps / BPS;  // 25%

// src/FoboRevenue.sol:267  recovery, 200 traders per call
function clearReferrers(address[] calldata traders) external onlyOwner
```

Recommendation

Keep the registrar key in a signer that cannot sign unattended batches, alert on any registerBy call that binds many traders to one referrer, and hold new referrers' share unclaimable for a short window so a bad batch can be cleared before it is paid.

● The Price Feed Is Checked For Staleness But Not For A Wrong Answer

Low-Risk *- Won't cause any problems, but can be adjusted for improvement*
[Status: Pending]

Error Code	Description
FOBO-L8	The feed handling is careful: it refuses a negative or zero answer, a missing or future timestamp, an answer older than the configured age, and it checks the sequencer uptime feed with a grace period. What it does not do is sanity check the size of the answer, so a feed that reports a fresh but badly wrong price, which is what happens when an aggregator pins at its own floor or ceiling, is used as if it were correct. Every limit order fill settles against that number, and the burner reads the same feed to decide whether a stablecoin route paid out fairly, so one wrong answer affects both contracts.

Code Snippet

```
// src/FoboLimitOrders.sol:202  _price
(, int256 answer,, uint256 updatedAt,) = p.feed.latestRoundData();
if (answer <= 0) revert FeedPriceInvalid();
// staleness and the sequencer are checked, the size is not

// src/FoboBurner.sol:174  the same feed bounds the peg check
uint256 ethUsd = IFoboLimitOrders(h.limitOrders()).ethUsd();
```

Recommendation

Store a plausible range per pay token and revert outside it, or compare the answer against the previous round and refuse a move larger than the feed's own deviation threshold allows in that time.

● A Limit Order Fills At Its Limit And The Keeper Keeps Everything Above It

Low-Risk *- Won't cause any problems, but can be adjusted for improvement*
[Status: Pending]

Error Code	Description
FOBO-L9	The keeper chooses when to fill an order, how much of it to fill, which pools to route through and what fee to take, and the only thing checked on chain is that the owner ends up with at least the limit they set. That check is well built, measured from the owner's actual balance change and rounded against the keeper, so an owner can never do worse than their limit. They also can never do better: if the market has moved far past the limit, the keeper can route the fill so the owner receives exactly their limit and keep the rest, which is the opposite of how people expect a limit order to behave.

Code Snippet

```
// src/FoboLimitOrders.sol:200  _allIn, the only bound on a fill
if (FullMath.mulDivRoundingUp(amountIn, price, unit)
        > FullMath.mulDiv(amountOut, order.limitPriceUsdE18, WAD))
    revert LimitNotMet();

// src/FoboLimitOrders.sol:156  the keeper picks all of it
function fill(uint256 id, uint256 amountIn, uint256 fee,
              Hop[] calldata route) external
```

Recommendation

Say in the order interface that a fill pays the limit price rather than the market price, or pass a share of the surplus back to the order's owner so the keeper is not the only party that benefits from a favourable move.

● A Quote Token Is Admitted On The Owner's Judgement With No On Chain Check

Medium-Risk *- Will likely cause problems and it is recommended to adjust*
[Status: Pending]

Error Code	Description
FOBO-M1	Allowing a new pair checks only that a start band exists and that the token answers decimals() with at least six. Nothing tests whether the quote takes a fee on transfer, rebases, or calls back into the caller during a transfer, and the code comment states that ruling those out is a manual review step. A quote that takes a fee would leave the PoolManager short when the factory settles, so the creator's launch buy reverts, and a launch with no creator buy still succeeds and leaves behind a pool that no one can trade through correctly. A quote that calls back on transfer also hands control to the payer in the middle of the launch callback.

Code Snippet

```
// src/FoboFactory.sol:325  setQuoteAllowed
if (allowed && quote != address(0) && quote != fobo) {
    if (c.bandLow == 0 && c.bandHigh == 0) revert NoBand();
    c.decimals = _decimalsOf(quote);   // the only token check
}
c.allowed = allowed;
```

Recommendation

Probe a candidate quote on chain before allowing it: send a known amount to the contract and require the received balance to match, and reject a token whose balance changes without a transfer. Keep the manual review as well, and publish which quotes are allowed.

● The Owner Can Withdraw Every Launch Fee And Sweep Held For The $FOBO Buyback

Medium-Risk *- Will likely cause problems and it is recommended to adjust*
[Status: Pending]

Error Code	Description
FOBO-M2	Every launch fee and every swept floor amount destined for the $FOBO buy-and-burn collects in the burner, and the owner can send all of it to any address at any time with no cap, no delay and no restriction on the recipient. The contract's own summary says nothing a keeper does can send anything anywhere but into $FOBO that is burned, which is true of keepers and not of the owner, so the one mechanism by which $FOBO is meant to accrue value rests on the owner choosing not to use this function. It is owner-only behind a two-step Safe and it cannot reach a pool's floor or anyone's wallet, which is why this is not rated higher.

Code Snippet

```
// src/FoboBurner.sol:256  emergencyWithdraw
function emergencyWithdraw(address token, address to)
    external onlyOwner
{
    uint256 claims =
        poolManager.balanceOf(address(this), c.toId());
    if (claims != 0) poolManager.unlock(...);  // the claims
    uint256 balance = c.balanceOfSelf();
    if (balance != 0) c.transfer(to, balance); // and the ETH
}

// src/FoboFactory.sol:143  where the launch fee goes
(bool sent,) = burner.call{value: spent}("");
```

Recommendation

Put this function behind a timelock long enough for holders to react, or restrict it to a recovery address fixed at deploy, and state plainly in the public material that the buyback funds are owner-recoverable.

Disclaimer

This audit report has been prepared by Coinsult's experts at the request of the client. In this audit, the results of the static analysis and the manual code review will be presented. The purpose of the audit is to see if the functions work as intended, and to identify potential security issues within the smart contract.

The information in this report should be used to understand the risks associated with the smart contract. This report can be used as a guide for the development team on how the contract could possibly be improved by remediating the issues that were identified.

Coinsult is not responsible if a project turns out to be a scam, rug-pull or honeypot. We only provide a detailed analysis for your own research.

Coinsult is not responsible for any financial losses. Nothing in this contract audit is financial advice, please do your own research.

The information provided in this audit is for informational purposes only and should not be considered investment advice. Coinsult does not endorse, recommend, support or suggest to invest in any project.

Coinsult can not be held responsible for when a project turns out to be a rug-pull, honeypot or scam.